Introduction and Overview

When learning to code, learners often find themselves with two common roadblocks. First, the scope of the language is too large, leaving the user overwhelmed about how to start or what techniques will work. Second, they become overly reliant on “good enough” techniques instead of learning the features needed to create more efficient, readable, and quick-to-implement approaches.

This project proposes creating a web app which will function as a Python learning environment. Its difference will be that certain features, such as loops, conditionals, or functions, can be switched off (i.e, they are no longer part of the syntax and cause a clear, readable error to be thrown), preventing users from relying too heavily on certain features and pushing them to understand new ones before they are misused. This will be aimed at beginner coders, and endeavours to improve upon the experience of learning Python simply through following tutorials or trying challenges in a standard code editor.

The web app will make use of restrictions in two main ways. First, by reducing the scope of language features available to the learner to exactly the features they know. This will be done by restricting all features at first, then slowly introducing more by way of simple coding challenges that the user will complete, proving their ability to understand the basics of that feature. Further, common built-in functions, such as sorted() or abs(), will be restricted until the user can supply their own implementations; this works similarly, providing the small challenge of implementing the function itself before it can be used.

The second method of restriction invites the learner to expand upon their basic understanding through research and challenge. The user can disallow features of the language that have previously been unlocked; this aims to challenge the user into finding new and sometimes unconventional methods of solving coding problems, which will be provided to them. As part of this, the web app will provide a function call visualiser. The aim of this is to invite the learner to think more deeply about how the features available to them increase the complexity of the programs they create.

Extension Features

The web app may also analyse files that the user uploads and probabilistically determine whether any of the features used within the program are being relied on too heavily. From this, it can then suggest a set of restrictions or challenges that the learner might find helpful. Part of the project will involve defining new types of exceptions for when different restricted features have been used; these will be made readable and concise, pointing exactly to where the “invalid” syntax is. To maintain consistency, the web app may also change the text of common, existing Python exceptions to be more in line with this style.

Starting Point

The base of the tool will be a code editor. Since creating my own would not add a substantial technical challenge to the project, I will be adapting an open-source one, namely,

https://github.com/isaacphysics/isaac-code-editor. It's a very limited editor, with only the ability to write, run, create, save, and download files. All other features in the finished product will be implemented by me, namely those described in the introductory section.

Further, as part of the project, I intend to use an open-source coding challenge website (https://github.com/exercism/python) for some of the challenges.

Success Criteria

1. The tool should be suitable for a complete beginner to use, avoiding unexplained technical language, and starting with challenges of an appropriate level.

2. The tool should allow full flexibility in which features are used; turning all additional features off should make it function as a simple code editor.

3. The tool's features should be useful for both complete beginners as well as those with some small experience in Python. At least three challenges should be provided for any user learning beginner-level features (looping, control flow, basic data structures, etc.)

4. I will test the run-time overhead added by using the tool. The tool should not add significant overhead, so that the programs run are still usable.

   To test criteria 1-3, I will perform an evaluation involving novice programmers. The evaluation will collect a range of metrics, such as time-to-complete certain challenges and usability metrics. For this evaluation, I will need to apply for ethics approval.

   To test criterion 4, I will perform a range of benchmarks on different execution size programs, comparing the difference in performance between running the program normally and through the tool.

Timeline

Period	Dates	Focus / Tasks
Michaelmas Term – Early Demo Implementation	**13/10–27/10**	Familiarisation with the codebase, research into existing approaches, and creation of design mockups for interfaces.
	28/10–10/11	Set up API, implement the ability to create restrictions, design interface for choosing restricted functions, restrict built-in functions.
	11/11–24/11	Allow users to write their own versions of built-in functions, set up restriction and unlock challenges.

	25/11–08/12	Determine correctness of user-written built-ins, buffer time for delays and reduced workload during busy term period.
Michaelmas Vacation – Completing Core Features	**Dec–Jan**	Develop a function call visualiser (2–3 weeks of dedicated work), include rest and revision period. All core features completed by end of vacation.
Lent Term – Extension Features and Evaluation	**19/01–01/02**	Implement analysis to detect overused features and recommend restrictions, begin readable exceptions, submit ethics application, and plan detailed evaluation.
	02/02–15/02	Implement readable exceptions, submit progress report.
	16/02–01/03	Finalise any delayed features.
	02/03–15/03	Conduct evaluations, analyse collected results.
Lent Vacation – First Draft of Write-Up	**Mar–Apr**	Produce first dissertation draft (evaluation, introduction, preparation, implementation), including a 2-week break and revision time.
Easter Term – Final Refinement	**Apr–May**	Continued redrafting and refinement of the dissertation with my supervisor until submission.

Resource Declaration

The project will be completed using my own laptop: a Framework 13 (2022). The specs are as follows:

- Intel i5-1240P
- 8GB RAM
- 1TB SSD

I will use GitHub and Overleaf for online back-ups of the program and write-up, respectively. Additionally, they will be saved locally on my device and backed up to a USB drive regularly. Should my laptop fail, I will have 2 methods of recovering my work so that I can continue to work on the project until the issue is fixed.

I accept full responsibility for this machine, and I have made contingency plans to protect myself against hardware and/or software failure, as detailed above.